

Take-Home Case Study

LLM-Powered Analytics MVP

Assignment Deadline: 5 days after receiving this case study

Background

Our product aims to help business users analyze structured data more easily. Users should be able to view key insights from a dataset and ask natural-language questions through an LLM-powered interface.

Your task is to build a lightweight MVP within 5 days. The goal is not to build a perfect production system, but to demonstrate how you research, design, build, validate, and explain a practical technical solution.

Objective

- Ingest and process a public dataset.
- Perform light-to-medium data engineering.
- Generate meaningful analytics and business insights.
- Provide a simple interface where users can ask analytical questions using an LLM.
- Explain the architecture, risks, trade-offs, and business value of your solution.

Dataset

You may choose one public dataset from Kaggle or another public source. Please choose a dataset from one of these categories:

- Retail / Sales / Customer Behavior - sales performance, customer segmentation, product/category analysis, or revenue trend.
- Customer Churn / Subscription - churn analysis, retention risk, customer behavior, or subscription performance.
- Operational / IoT / Time-Series - anomaly detection, equipment monitoring, usage trend, or operational performance.

You must explain:

- Why you chose the dataset.
- What business problem the dataset represents.
- What assumptions and limitations exist in the dataset.
- What kind of business questions users may ask from the dataset.

Mandatory Requirements

1. Data Engineering Pipeline

- Data ingestion from CSV, API, or local file.
- Data cleaning and transformation.
- Data quality checks.
- Basic schema design or data modeling.
- Creation of an analytics-ready dataset.

- Documentation of the transformation logic.

Expected flow: Raw Data -> Cleaned Data -> Analytics-Ready Data. A stronger solution may include Raw Data -> Staging Layer -> Transformation Layer -> Curated Analytics Layer.

2. Analytics Layer

- Create an analytics layer that answers at least 5 meaningful business questions.
- Examples: overall trend, top segment, anomalies/outliers, factors influencing target metrics, and recommendations.
- You may use Python notebook, Streamlit, Superset, Metabase, Power BI, Looker Studio, a custom dashboard, SQL-based analytics, or another tool.
- You must explain why you chose that approach or platform.

3. LLM-Powered Prompt Interface

- Build a simple prompt interface where users can ask natural-language questions about the processed dataset or analytics results.
- Minimum expected flow: User asks a question -> System retrieves or analyzes relevant data -> LLM returns an answer grounded in the data.
- The LLM answer must be based on the dataset, analytics output, SQL result, dataframe result, or precomputed summary.
- The LLM should not answer freely without grounding from the data.

You must explain how the LLM accesses the data, how hallucination is reduced, how answers are validated, what happens when a question cannot be answered, and what security/privacy risks should be considered.

4. MVP Interface

- One dashboard or analytics page.
- One prompt box for natural-language questions.
- Clear output from the LLM.
- Short explanation of how the answer was generated.

A better version may include suggested questions, data/metric references, and error handling. An excellent version may include a simple agent that can query data, read summaries, generate charts, or suggest follow-up analysis.

Optional Plus Points

- Simple AI agent workflow.
- Suggested follow-up questions.
- SQL or dataframe tool-calling.
- Data quality monitoring.
- LLM response evaluation or hallucination checks.
- Dockerized setup.
- Architecture diagram.
- Simple deployment link.
- Exportable insight summary.
- Security/privacy design, such as masking, access control assumptions, or safe logging.

These are optional. A simple but well-scoped and well-explained MVP is better than an overbuilt but unfinished system.

Deliverables

1. Git Repository

- Source code.
- README.
- Setup instructions.
- Dataset reference.
- Assumptions.
- How to run the MVP.
- Known limitations.

2. Architecture Document

- Maximum 3-5 pages.
- Problem statement.
- Architecture diagram.
- Data flow.
- Technology choices and trade-offs.
- Security/privacy considerations.
- What is included in the POC/MVP.
- What is not production-ready yet.
- Future improvement plan.

3. Demo Video

- Maximum 7-10 minutes.
- Dataset and business problem.
- Data pipeline.
- Analytics output.
- LLM-powered prompt interface.
- Risks and limitations.
- What you would improve with more time.

4. Short Presentation

- Maximum 5 slides.
- Suggested structure: problem and dataset, architecture and data flow, analytics insights, LLM/prompt interface, trade-offs/risks/next steps.

Suggested Timeline

You have 5 days to complete the assignment. You may adjust the timeline based on your approach.

Day	Focus
Day 1	Dataset selection, problem framing, success criteria, initial architecture
Day 2	Data ingestion, cleaning, and data quality checks
Day 3	Data modeling, analytics-ready layer, and initial analytics
Day 4	Analytics/dashboard development and LLM prompt interface with grounding mechanism
Day 5	Testing, documentation, risk/trade-off analysis, final demo preparation and submission

Evaluation Criteria

Area	Weight
Problem framing & business understanding	10%
Data engineering quality	15%
Analytics quality	15%
Architecture & technical design	15%
LLM/prompt interface	15%
Risk, security & hallucination awareness	10%
Pragmatism & anti-over-engineering	10%
Communication & documentation	10%

What We Care About

We care about how you think, not just what tools you use. A strong submission should show that you can:

- Understand a business problem from data.
- Build a simple but reliable data pipeline.
- Generate meaningful insights, not just charts.
- Design a practical MVP architecture.
- Use LLM in a grounded and responsible way.
- Identify risks early.
- Explain trade-offs clearly.
- Avoid over-engineering.
- Communicate complex technical concepts simply.

Important Notes

Please do not:

- Spend too much time making the UI beautiful.
- Build a full production system.
- Use LLM to answer without grounding from the data.
- Submit only a notebook without an MVP interface.
- Submit only a dashboard without explaining the data pipeline.
- Only call OpenAI/Gemini API and claim it as an AI analytics platform.
- Hide limitations or assumptions.
- Overbuild features that do not support the core objective.

This is an MVP case study. It is acceptable to make shortcuts as long as you clearly explain the trade-offs and limitations.

Final Submission

- Git repository link.
- Demo video link.
- Architecture document.
- Short presentation.

Deadline: 5 days after receiving this assignment.